```
#!/usr/bin/env python3
# ================================================================================
#  ADAPTIVE OFDM WIRELESS COMMUNICATION SYSTEM - FULL SIMULATION
# ================================================================================
#  Author : DHANUSH P(MTECH COMMUNICATION ENGINEERING)

#  Description:
#     End-to-end software simulation of a modern OFDM wireless link.
#
#     TX chain : Text -> Bits -> Adaptive M-ary Modulation (BPSK/QPSK/16QAM/64
#                -> OFDM (pilot insertion, IFFT, Cyclic Prefix)
#     CHANNEL  : Frequency-selective multipath Rayleigh fading + AWGN
#     RX chain : Remove CP -> FFT -> LS pilot-based channel estimation
#                -> linear interpolation -> MMSE equalization
#                -> Demodulation -> Bits -> Text
#
#     Performance analysis : BER, SER, EVM, Throughput, Spectral Efficiency
#     Adaptive modulation   : modulation order selected from operating SNR
#
#  Run:
#     python3 adaptive_ofdm.py
#
#  All results (plots) are saved as PNG files in the current directory
#  (or the folder given by --outdir).
# ================================================================================

import os
import sys
import argparse
import numpy as np
import matplotlib

def _running_in_notebook() -> bool:
    """Detect Jupyter / Colab / IPython so we know whether inline plot
    display (plt.show()) will actually render something."""
    try:
        from IPython import get_ipython
        shell = get_ipython()
        return shell is not None
    except Exception:
        return False
IN_NOTEBOOK = _running_in_notebook()
if not IN_NOTEBOOK:
    # Headless-safe backend for plain scripts / servers with no display.
    matplotlib.use("Agg")
else:
    # We're inside a live IPython/Jupyter/Colab kernel: force the inline
    # backend explicitly so plt.show() reliably renders figures under the
    # cell, regardless of any prior backend configuration.
    try:
        from IPython import get_ipython
        get_ipython().run_line_magic("matplotlib", "inline")
    except Exception:
        pass
```

```
import matplotlib.pyplot as plt

np.set_printoptions(precision=4, suppress=True)

# ============================================================================
# SECTION 1 : TEXT  <->  BITS
# ============================================================================

def text_to_bits(text: str) -> np.ndarray:
    """Convert a text string into a 1-D array of bits (uint8), UTF-8 encoded."
    data = text.encode("utf-8")
    if len(data) == 0:
        return np.array([], dtype=np.uint8)
    return np.unpackbits(np.frombuffer(data, dtype=np.uint8))


def bits_to_text(bits: np.ndarray) -> str:
    """Convert a 1-D array of bits (uint8, 0/1) back into a text string."""
    n_bytes = len(bits) // 8
    if n_bytes == 0:
        return ""
    bits = np.asarray(bits[: n_bytes * 8], dtype=np.uint8)
    byte_arr = np.packbits(bits)
    return byte_arr.tobytes().decode("utf-8", errors="replace")


def build_bitstream(text: str) -> np.ndarray:
    """
    Build the actual bitstream sent over the air:
        [ 32-bit header = number of payload bits ] + [ payload bits ]
    The header lets the receiver know exactly how many bits are real
    text data versus zero-padding added later to fill OFDM symbols.
    """
    payload = text_to_bits(text)
    n = len(payload)
    header = np.array([(n >> (31 - i)) & 1 for i in range(32)], dtype=np.uint8
    return np.concatenate([header, payload]).astype(np.uint8)


def parse_bitstream(bits: np.ndarray):
    """Inverse of build_bitstream(). Returns (payload_bits, n_payload_bits)."""
    if len(bits) < 32:
        return np.array([], dtype=np.uint8), 0
    header = bits[:32]
    n = 0
    for b in header:
        n = (n << 1) | int(b)
    n = max(0, min(n, len(bits) - 32))
    payload = bits[32: 32 + n]
    return payload, n


# ============================================================================
# SECTION 2 : DIGITAL MODULATION  (BPSK, QPSK, 16-QAM, 64-QAM)
#             Generic square-QAM Gray-coded mapper/demapper.
# ============================================================================
```

```
MODULATIONS = {
    "BPSK":  1,
    "QPSK":  2,
    "16QAM": 4,
    "64QAM": 6,
}
MOD_ORDER = list(MODULATIONS.keys())


def _gray_encode(n: int) -> int:
    return n ^ (n >> 1)


def _gray_decode(g: int) -> int:
    mask = g
    b = g
    while mask:
        mask >>= 1
        b ^= mask
    return b


def _bits_to_int(bits) -> int:
    val = 0
    for b in bits:
        val = (val << 1) | int(b)
    return val


def _int_to_bits(val: int, k: int) -> np.ndarray:
    return np.array([(val >> (k - 1 - i)) & 1 for i in range(k)], dtype=np.uin


def _pam_modulate(bits_group: np.ndarray, k: int) -> float:
    """k Gray-coded bits -> one PAM level in {-(L-1), ..., -1, 1, ..., L-1}.""
    g = _bits_to_int(bits_group)
    idx = _gray_decode(g)
    L = 2 ** k
    return 2 * idx - (L - 1)


def _pam_demodulate(level_est: float, k: int) -> np.ndarray:
    """Nearest-level hard decision -> k Gray-coded bits."""
    L = 2 ** k
    idx = int(round((level_est + (L - 1)) / 2))
    idx = max(0, min(L - 1, idx))
    g = _gray_encode(idx)
    return _int_to_bits(g, k)


def modulate(bits: np.ndarray, scheme: str) -> np.ndarray:
    """
    Map a bit array (length MUST be a multiple of bits/symbol) to unit-average
    -energy complex symbols for the requested modulation scheme.
    """
    k = MODULATIONS[scheme]
```

```
    bits = np.asarray(bits, dtype=np.uint8)
    n_sym = len(bits) // k
    bits = bits[: n_sym * k].reshape(n_sym, k)

    if scheme == "BPSK":
        symbols = np.where(bits[:, 0] == 1, 1.0, -1.0).astype(complex)
        return symbols

    kd = k // 2                         # bits per I / Q dimension
    L = 2 ** kd
    norm = np.sqrt(2 * (L ** 2 - 1) / 3)   # average-energy normalization
    symbols = np.empty(n_sym, dtype=complex)
    for i in range(n_sym):
        bi, bq = bits[i, :kd], bits[i, kd:]
        I = _pam_modulate(bi, kd)
        Q = _pam_modulate(bq, kd)
        symbols[i] = (I + 1j * Q) / norm
    return symbols


def demodulate(symbols: np.ndarray, scheme: str) -> np.ndarray:
    """Hard-decision demapper: complex symbols -> Gray-coded bits."""
    k = MODULATIONS[scheme]
    if scheme == "BPSK":
        return (np.real(symbols) >= 0).astype(np.uint8)

    kd = k // 2
    L = 2 ** kd
    norm = np.sqrt(2 * (L ** 2 - 1) / 3)
    out = np.zeros(len(symbols) * k, dtype=np.uint8)
    for i, s in enumerate(symbols):
        I = np.real(s) * norm
        Q = np.imag(s) * norm
        bi = _pam_demodulate(I, kd)
        bq = _pam_demodulate(Q, kd)
        out[i * k: i * k + kd] = bi
        out[i * k + kd: i * k + k] = bq
    return out


def ideal_constellation(scheme: str) -> np.ndarray:
    """Return all ideal constellation points of a scheme (for plotting)."""
    k = MODULATIONS[scheme]
    n = 2 ** k
    all_bits = np.array([[int(b) for b in format(i, f"0{k}b")] for i in range(
                         dtype=np.uint8).reshape(-1)
    return modulate(all_bits, scheme)


# ============================================================================
# SECTION 3 : ADAPTIVE MODULATION SELECTION
# ============================================================================

# SNR (dB) thresholds -> modulation order.
# Higher operating SNR permits the use of higher-order constellations.
# Thresholds are predefined simulation parameters.
ADAPTIVE_THRESHOLDS = [
```

```
    (5.0, "BPSK"),
    (12.0, "QPSK"),
    (20.0, "16QAM"),
    (float("inf"), "64QAM"),
]

def choose_modulation_adaptive(snr_db):
    for threshold, modulation in ADAPTIVE_THRESHOLDS:
        if snr_db < threshold:
            return modulation
    return "64QAM"


# ================================================================================
# SECTION 4 : OFDM SYSTEM (framing, pilots, IFFT/FFT, cyclic prefix)
# ================================================================================

class OFDMSystem:
    def __init__(self, n_fft=64, n_cp=16, pilot_spacing=4,
                 pilot_value=(1 + 1j) / np.sqrt(2), fs=20e6):
        self.n_fft = n_fft
        self.n_cp = n_cp
        self.pilot_indices = np.arange(0, n_fft, pilot_spacing)
        self.data_indices = np.array(
            [i for i in range(n_fft) if i not in set(self.pilot_indices)])
        self.n_pilot = len(self.pilot_indices)
        self.n_data = len(self.data_indices)
        self.pilot_value = pilot_value
        self.fs = fs                        # sample rate (Hz)
        self.delta_f = fs / n_fft           # subcarrier spacing (Hz)
        self.ts = 1.0 / fs                  # sample period (s)

    # ---------------- transmitter side ----------------
    def build_frame(self, data_symbols: np.ndarray):
        """Place data symbols + pilots on an n_fft x n_ofdm_syms resource grid
        n_needed = int(np.ceil(len(data_symbols) / self.n_data)) * self.n_data
        n_needed = max(n_needed, self.n_data)
        pad_len = n_needed - len(data_symbols)
        if pad_len > 0:
            data_symbols = np.concatenate(
                [data_symbols, np.zeros(pad_len, dtype=complex)])
        n_ofdm_syms = n_needed // self.n_data

        grid = np.zeros((self.n_fft, n_ofdm_syms), dtype=complex)
        grid[self.pilot_indices, :] = self.pilot_value
        data_mat = data_symbols.reshape(n_ofdm_syms, self.n_data).T
        grid[self.data_indices, :] = data_mat
        return grid, pad_len, n_ofdm_syms

    def ifft_and_cp(self, grid: np.ndarray) -> np.ndarray:
        """OFDM modulate: per-symbol IFFT + cyclic prefix, then serialize."""
        time_grid = np.fft.ifft(grid, axis=0) * np.sqrt(self.n_fft)
        cp = time_grid[-self.n_cp:, :]
        with_cp = np.vstack([cp, time_grid])           # (n_fft+n_cp, n_syms)
        serial = with_cp.flatten(order="F")             # time-sequential
        return serial
```

```
    # ---------------- receiver side ----------------
    def remove_cp_and_fft(self, rx_serial: np.ndarray, n_ofdm_syms: int) -> np
        sym_len = self.n_fft + self.n_cp
        needed = sym_len * n_ofdm_syms
        rx_serial = rx_serial[:needed]
        rx_mat = rx_serial.reshape(n_ofdm_syms, sym_len).T   # (sym_len, n_sym
        rx_no_cp = rx_mat[self.n_cp:, :]
        R = np.fft.fft(rx_no_cp, axis=0) / np.sqrt(self.n_fft)
        return R


# ============================================================================
# SECTION 5 : CHANNEL MODEL  (frequency-selective Rayleigh fading + AWGN)
# ============================================================================

def generate_multipath_channel(num_taps=6, decay=1.0, seed=None) -> np.ndarray
    """
    Generate a random complex-Gaussian tapped-delay-line channel with an
    exponentially decaying power-delay-profile (Rayleigh fading per tap).
    Total average tap power is normalized to 1.
    """
    rng = np.random.default_rng(seed)
    powers = np.exp(-decay * np.arange(num_taps))
    powers = powers / powers.sum()
    taps = (rng.standard_normal(num_taps) + 1j * rng.standard_normal(num_taps)
    taps = taps * np.sqrt(powers / 2)
    return taps


def apply_channel(signal: np.ndarray, taps: np.ndarray) -> np.ndarray:
    """Convolve the transmitted signal with the multipath channel."""
    out = np.convolve(signal, taps, mode="full")[: len(signal)]
    return out


def add_awgn(signal: np.ndarray, snr_db: float, seed=None):
    """Add complex AWGN to reach the requested (average) SNR in dB."""
    rng = np.random.default_rng(seed)
    sig_power = np.mean(np.abs(signal) ** 2)
    snr_linear = 10 ** (snr_db / 10.0)
    noise_power = sig_power / snr_linear
    noise = np.sqrt(noise_power / 2) * (
        rng.standard_normal(len(signal)) + 1j * rng.standard_normal(len(signal
    return signal + noise, noise_power


# ============================================================================
# SECTION 6 : CHANNEL ESTIMATION (LS + interpolation) AND MMSE EQUALIZATION
# ============================================================================

def ls_channel_estimate(R: np.ndarray, ofdm: OFDMSystem) -> np.ndarray:
    """
    Least-Squares channel estimate at pilot subcarriers, averaged across all
    OFDM symbols in the frame (quasi-static / block-fading assumption).
    """
    pilot_rx = R[ofdm.pilot_indices, :]                # n_pilot x n_syms
```

```
    H_ls = pilot_rx / ofdm.pilot_value
    return np.mean(H_ls, axis=1)                          # n_pilot


def interpolate_channel(H_pilot_avg: np.ndarray, ofdm: OFDMSystem) -> np.ndarra
    """Linear interpolation of the pilot-based estimate across all subcarriers
    x = ofdm.pilot_indices
    xi = np.arange(ofdm.n_fft)
    re = np.interp(xi, x, H_pilot_avg.real)
    im = np.interp(xi, x, H_pilot_avg.imag)
    return re + 1j * im


def mmse_equalize(R: np.ndarray, H_est: np.ndarray, noise_var: float) -> np.nda
    """
    Per-subcarrier MMSE equalizer:
        W(k) = H*(k) / (|H(k)|^2 + sigma_n^2)
    applied to every OFDM symbol column of R.
    """
    H = H_est[:, None]
    W = np.conj(H) / (np.abs(H) ** 2 + noise_var)
    return W * R


# ============================================================================
# SECTION 7 : PERFORMANCE METRICS
# ============================================================================

def compute_ber(tx_bits: np.ndarray, rx_bits: np.ndarray):
    n = min(len(tx_bits), len(rx_bits))
    if n == 0:
        return 0.0, 0
    errors = int(np.sum(tx_bits[:n] != rx_bits[:n]))
    return errors / n, errors


def compute_ser(scheme: str, tx_bits: np.ndarray, rx_bits: np.ndarray):
    k = MODULATIONS[scheme]
    n = (min(len(tx_bits), len(rx_bits)) // k) * k
    if n == 0:
        return 0.0, 0
    tx_syms = tx_bits[:n].reshape(-1, k)
    rx_syms = rx_bits[:n].reshape(-1, k)
    sym_err = int(np.sum(np.any(tx_syms != rx_syms, axis=1)))
    return sym_err / tx_syms.shape[0], sym_err


def compute_evm(tx_symbols: np.ndarray, rx_symbols: np.ndarray) -> float:
    n = min(len(tx_symbols), len(rx_symbols))
    if n == 0:
        return 0.0
    err = rx_symbols[:n] - tx_symbols[:n]
    denom = np.mean(np.abs(tx_symbols[:n]) ** 2)
    if denom == 0:
        return 0.0
    evm_rms = np.sqrt(np.mean(np.abs(err) ** 2) / denom)
    return float(evm_rms * 100.0)          # percent
```

```
def compute_throughput_se(scheme: str, ofdm: OFDMSystem, n_ofdm_syms: int,
                          payload_bits: int):
    """
    throughput [bits/s]         : useful payload bits / total air-time
    spectral efficiency [b/s/Hz]: throughput / occupied bandwidth
    Air-time and bandwidth both include the CP / pilot overhead, so both
    metrics correctly reflect real system efficiency losses.
    """
    t_symbol = (ofdm.n_fft + ofdm.n_cp) * ofdm.ts
    total_time = n_ofdm_syms * t_symbol
    throughput = payload_bits / total_time if total_time > 0 else 0.0
    bandwidth = ofdm.n_fft * ofdm.delta_f
    se = throughput / bandwidth if bandwidth > 0 else 0.0
    return throughput, se


# ============================================================================
# SECTION 7B : LIGHTWEIGHT FEC (repetition code) + BLOCK INTERLEAVER
# ============================================================================
#  NOTE ON REALISM:
#  Uncoded transmission over a frequency-selective Rayleigh fading channel
#  can experience burst errors when some subcarriers fall into deep fades.
#  Adaptive modulation improves spectral efficiency by selecting a modulation
#  order according to the operating SNR, while channel coding and interleaving
#  improve reliability.
#
#  The repetition code + block interleaver used here are lightweight educationa
#  mechanisms for improving reliability in the live text demonstration.
#  They do not guarantee error-free reconstruction under severe fading/noise.
#
#  The BER/SER/EVM Monte-Carlo sweep remains UNCODED so that raw modulation
#  performance can be compared across BPSK, QPSK, 16-QAM and 64-QAM.
# ============================================================================

def repetition_encode(bits: np.ndarray, rep: int = 3) -> np.ndarray:
    if rep <= 0 or rep % 2 == 0:
        raise ValueError("Repetition factor must be a positive odd integer.")
    return np.repeat(bits, rep)


def repetition_decode(bits: np.ndarray, rep: int = 3) -> np.ndarray:
    if rep <= 0 or rep % 2 == 0:
        raise ValueError("Repetition factor must be a positive odd integer.")

    n = len(bits) // rep
    bits = bits[: n * rep].reshape(n, rep)
    votes = np.sum(bits, axis=1)
    return (votes > (rep / 2.0)).astype(np.uint8)


def block_interleave(bits: np.ndarray, seed: int = 2024):
    """Pseudo-random block interleaver: scatters repeated copies of the same
    bit across widely separated OFDM subcarriers/symbols so a single deep
    fade cannot wipe out all copies of one bit at once."""
    rng = np.random.default_rng(seed)
```

```
    perm = rng.permutation(len(bits))
    return bits[perm], perm


def block_deinterleave(bits: np.ndarray, perm: np.ndarray) -> np.ndarray:
    out = np.empty_like(bits)
    out[perm] = bits
    return out


# ============================================================================
# SECTION 8 : FULL END-TO-END PIPELINE (single message run)
# ============================================================================

class OFDMResult:
    """Simple container for everything produced by one pipeline run."""
    pass


def run_ofdm_system(text: str, scheme="QPSK", snr_db=20.0, ofdm: OFDMSystem = N
                    channel_taps=None, num_taps=6, adaptive=False, seed=None,
                    use_fec=True, rep=5):

    if ofdm is None:
        ofdm = OFDMSystem()

    if adaptive:
        scheme = choose_modulation_adaptive(snr_db)

    k = MODULATIONS[scheme]

    # ---- 1. Text -> bitstream (with 32-bit length header) ----
    tx_bits_msg = build_bitstream(text)          # header + payload (uncoded)

    # ---- 1b. Optional FEC: repetition code + block interleaving ----
    if use_fec:
        coded_bits = repetition_encode(tx_bits_msg, rep=rep)
        coded_bits, interleave_perm = block_interleave(coded_bits, seed=2024)
    else:
        coded_bits = tx_bits_msg
        interleave_perm = None

    tx_bits_full = coded_bits
    # pad with zeros so length is an exact multiple of bits/symbol
    pad_bits = (-len(tx_bits_full)) % k
    if pad_bits:
        tx_bits_full = np.concatenate(
            [tx_bits_full, np.zeros(pad_bits, dtype=np.uint8)])

    # ---- 2. Modulation ----
    tx_symbols = modulate(tx_bits_full, scheme)

    # ---- 3. OFDM framing (pilots + IFFT + CP) ----
    grid, pad_len, n_ofdm_syms = ofdm.build_frame(tx_symbols)
    tx_serial = ofdm.ifft_and_cp(grid)

    #     4. Channel: frequency-selective Rayleigh fading + AWGN
```

```
    # ---- 4. Channel: frequency-selective Rayleigh fading + AWGN
    if channel_taps is None:
        channel_seed = None if seed is None else seed + 10001
        channel_taps = generate_multipath_channel(
            num_taps=num_taps,
            seed=channel_seed
        )

    rx_faded = apply_channel(tx_serial, channel_taps)

    noise_seed = None if seed is None else seed + 20001
    rx_noisy, noise_var = add_awgn(
        rx_faded,
        snr_db,
        seed=noise_seed
    )

    # ---- 5. Receiver: remove CP, FFT ----
    R = ofdm.remove_cp_and_fft(rx_noisy, n_ofdm_syms)

    # ---- 6. Channel estimation (LS @ pilots + interpolation) ----
    H_pilot_avg = ls_channel_estimate(R, ofdm)
    H_est = interpolate_channel(H_pilot_avg, ofdm)

    # ---- 7. MMSE equalization ----
    X_hat = mmse_equalize(R, H_est, noise_var)

    # ---- 8. Extract data subcarriers back into a symbol stream ----
    rx_data_symbols = X_hat[ofdm.data_indices, :].T.flatten()
    rx_data_symbols = rx_data_symbols[: len(tx_symbols)]   # drop OFDM padding

    # ---- 9. Demodulation ----
    rx_bits_full = demodulate(rx_data_symbols, scheme)
    rx_bits_full = rx_bits_full[: len(tx_bits_full)]

    # ---- 9b. Raw (pre-FEC) channel BER/SER, measured on the CODED bits ----
    ber, n_bit_err = compute_ber(tx_bits_full, rx_bits_full)
    ser, n_sym_err = compute_ser(scheme, tx_bits_full, rx_bits_full)
    evm = compute_evm(tx_symbols, rx_data_symbols)

    # ---- 10. Optional FEC decoding: de-interleave + majority vote ----
    if use_fec:
        coded_len = len(coded_bits)
        rx_coded = rx_bits_full[:coded_len]
        rx_deint = block_deinterleave(rx_coded, interleave_perm)
        rx_bits_msg = repetition_decode(rx_deint, rep=rep)
        rx_bits_msg = rx_bits_msg[: len(tx_bits_msg)]
    else:
        rx_bits_msg = rx_bits_full[: len(tx_bits_msg)]

    # ---- 11. Recover text using the length header ----
    rx_payload, n_payload = parse_bitstream(rx_bits_msg)
    recovered_text = bits_to_text(rx_payload)
    post_fec_ber, post_fec_err = compute_ber(tx_bits_msg, rx_bits_msg)
    payload_bits = len(text_to_bits(text))

    throughput, se = compute_throughput_se(
```

```
        scheme,
        ofdm,
        n_ofdm_syms,
        payload_bits
    )
    H_true = np.fft.fft(channel_taps, ofdm.n_fft)

    res = OFDMResult()
    res.scheme = scheme
    res.original_text = text
    res.recovered_text = recovered_text
    res.text_match = (recovered_text == text)
    res.tx_bits = tx_bits_full
    res.rx_bits = rx_bits_full
    res.tx_symbols = tx_symbols
    res.rx_symbols = rx_data_symbols
    res.ber = ber
    res.n_bit_err = n_bit_err
    res.ser = ser
    res.n_sym_err = n_sym_err
    res.evm_percent = evm
    res.throughput_bps = throughput
    res.spectral_eff = se
    res.channel_taps = channel_taps
    res.H_true = H_true
    res.H_est = H_est
    res.snr_db = snr_db
    res.noise_var = noise_var
    res.n_ofdm_syms = n_ofdm_syms
    res.ofdm = ofdm
    res.use_fec = use_fec
    res.rep = rep
    res.post_fec_ber = post_fec_ber
    res.post_fec_err = post_fec_err
    return res


# ============================================================================
# SECTION 9 : MONTE-CARLO BER / SER / EVM SWEEP OVER SNR
# ============================================================================

def run_ber_sweep(snr_range, schemes, n_trials=15, n_bits=6000, ofdm=None, see
    """
    For each modulation scheme and each SNR point, run several independent
    random-bit trials (fresh random channel + noise each time) and average
    BER / SER / EVM. Returns a nested dict: results[scheme]['ber'/'ser'/'evm']
    """
    if ofdm is None:
        ofdm = OFDMSystem()

    results = {s: {"ber": [], "ser": [], "evm": []} for s in schemes}

    for scheme in schemes:
        k = MODULATIONS[scheme]
        n_bits_use = (n_bits // k) * k
        for snr in snr_range:
            ber_acc = ser_acc = evm_acc = 0.0
```

```
            for t in range(n_trials):
                scheme_id = list(MODULATIONS.keys()).index(scheme)
                snr_id = int(round((snr + 1000) * 10))
                seed = seed0 + scheme_id * 100000 + snr_id * 100 + t
                rng = np.random.default_rng(seed)
                bits = rng.integers(0, 2, size=n_bits_use).astype(np.uint8)

                tx_symbols = modulate(bits, scheme)
                grid, pad_len, n_syms = ofdm.build_frame(tx_symbols)
                tx_serial = ofdm.ifft_and_cp(grid)

                taps = generate_multipath_channel(num_taps=6, seed=seed)
                rx_faded = apply_channel(tx_serial, taps)
                rx_noisy, noise_var = add_awgn(rx_faded, snr, seed=seed)

                R = ofdm.remove_cp_and_fft(rx_noisy, n_syms)
                H_pilot_avg = ls_channel_estimate(R, ofdm)
                H_est = interpolate_channel(H_pilot_avg, ofdm)
                X_hat = mmse_equalize(R, H_est, noise_var)

                rx_syms = X_hat[ofdm.data_indices, :].T.flatten()[: len(tx_syml
                rx_bits = demodulate(rx_syms, scheme)[: len(bits)]

                ber, _ = compute_ber(bits, rx_bits)
                ser, _ = compute_ser(scheme, bits, rx_bits)
                evm = compute_evm(tx_symbols, rx_syms)

                ber_acc += ber
                ser_acc += ser
                evm_acc += evm

            results[scheme]["ber"].append(ber_acc / n_trials)
            results[scheme]["ser"].append(ser_acc / n_trials)
            results[scheme]["evm"].append(evm_acc / n_trials)

    return results


def build_adaptive_curve(snr_range, sweep_results):
    """Pick, at each SNR point, the BER/SER/EVM of the scheme adaptive
    modulation would have chosen at that SNR."""
    adaptive = {"scheme": [], "ber": [], "ser": [], "evm": []}
    for i, snr in enumerate(snr_range):
        scheme = choose_modulation_adaptive(snr)
        adaptive["scheme"].append(scheme)
        adaptive["ber"].append(sweep_results[scheme]["ber"][i])
        adaptive["ser"].append(sweep_results[scheme]["ser"][i])
        adaptive["evm"].append(sweep_results[scheme]["evm"][i])
    return adaptive


# ============================================================================
# SECTION 10 : PLOTTING
# ============================================================================

def plot_constellation(res: OFDMResult, outpath: str):
    ideal = ideal_constellation(res.scheme)
```

```
    fig, axes = plt.subplots(1, 2, figsize=(10, 5))

    axes[0].scatter(res.tx_symbols.real, res.tx_symbols.imag, s=10, alpha=0.6)
    axes[0].scatter(ideal.real, ideal.imag, marker="x", color="red", s=80,
                    label="Ideal points")
    axes[0].set_title(f"Transmitted constellation ({res.scheme})")
    axes[0].set_xlabel("In-phase")
    axes[0].set_ylabel("Quadrature")
    axes[0].axis("equal")
    axes[0].grid(True, alpha=0.3)
    axes[0].legend()

    axes[1].scatter(res.rx_symbols.real, res.rx_symbols.imag, s=10, alpha=0.5,
                    color="orange")
    axes[1].scatter(ideal.real, ideal.imag, marker="x", color="red", s=80,
                    label="Ideal points")
    axes[1].set_title(f"Received constellation after MMSE eq.\n"
                      f"SNR={res.snr_db:.1f} dB, EVM={res.evm_percent:.2f}%")
    axes[1].set_xlabel("In-phase")
    axes[1].set_ylabel("Quadrature")
    axes[1].axis("equal")
    axes[1].grid(True, alpha=0.3)
    axes[1].legend()

    fig.tight_layout()
    fig.savefig(outpath, dpi=150)
    plt.show()
    plt.close(fig)


def plot_channel_response(res: OFDMResult, outpath: str):
    n_fft = res.ofdm.n_fft
    freqs = np.arange(n_fft)
    fig, axes = plt.subplots(2, 1, figsize=(9, 7), sharex=True)

    axes[0].plot(freqs, np.abs(res.H_true), label="True |H(f)|", linewidth=2)
    axes[0].plot(freqs, np.abs(res.H_est), "--", label="Estimated |H(f)| (LS+i
                 linewidth=2)
    axes[0].scatter(res.ofdm.pilot_indices,
                    np.abs(res.H_true[res.ofdm.pilot_indices]),
                    color="red", zorder=5, label="Pilot subcarriers")
    axes[0].set_ylabel("|H(f)|")
    axes[0].set_title("Channel Frequency Response: True vs. Estimated")
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    axes[1].plot(freqs, np.angle(res.H_true), label="True phase")
    axes[1].plot(freqs, np.angle(res.H_est), "--", label="Estimated phase")
    axes[1].set_xlabel("Subcarrier index")
    axes[1].set_ylabel("Phase (rad)")
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    fig.tight_layout()
    fig.savefig(outpath, dpi=150)
    plt.show()
```

```
    plt.close(fig)


def plot_ber_ser_evm_sweep(snr_range, sweep_results, adaptive, outpath: str):
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))

    markers = {"BPSK": "o", "QPSK": "s", "16QAM": "^", "64QAM": "d"}

    for scheme in sweep_results:
        ber = np.array(sweep_results[scheme]["ber"])
        ber = np.clip(ber, 1e-6, None)
        axes[0].semilogy(snr_range, ber, marker=markers.get(scheme, "o"),
                         label=scheme)
    ada_ber = np.clip(np.array(adaptive["ber"]), 1e-6, None)
    axes[0].semilogy(snr_range, ada_ber, "k*--", label="Adaptive", linewidth=2
                     markersize=10)
    axes[0].set_xlabel("SNR (dB)")
    axes[0].set_ylabel("Bit Error Rate")
    axes[0].set_title("BER vs SNR")
    axes[0].grid(True, which="both", alpha=0.3)
    axes[0].legend()

    for scheme in sweep_results:
        ser = np.array(sweep_results[scheme]["ser"])
        ser = np.clip(ser, 1e-6, None)
        axes[1].semilogy(snr_range, ser, marker=markers.get(scheme, "o"),
                         label=scheme)
    ada_ser = np.clip(np.array(adaptive["ser"]), 1e-6, None)
    axes[1].semilogy(snr_range, ada_ser, "k*--", label="Adaptive", linewidth=2
                     markersize=10)
    axes[1].set_xlabel("SNR (dB)")
    axes[1].set_ylabel("Symbol Error Rate")
    axes[1].set_title("SER vs SNR")
    axes[1].grid(True, which="both", alpha=0.3)
    axes[1].legend()

    for scheme in sweep_results:
        evm = np.array(sweep_results[scheme]["evm"])
        axes[2].plot(snr_range, evm, marker=markers.get(scheme, "o"), label=sc
    axes[2].plot(snr_range, adaptive["evm"], "k*--", label="Adaptive", linewid
                 markersize=10)
    axes[2].set_xlabel("SNR (dB)")
    axes[2].set_ylabel("EVM (%)")
    axes[2].set_title("EVM vs SNR")
    axes[2].grid(True, alpha=0.3)
    axes[2].legend()

    fig.tight_layout()
    fig.savefig(outpath, dpi=150)
    plt.show()
    plt.close(fig)


def plot_throughput_se(snr_range, schemes, ofdm, outpath: str):
    """Reference throughput and spectral efficiency versus SNR
    for fixed modulation schemes and adaptive modulation.
    A fixed 50-OFDM-symbol frame is used for comparison.
```

```
    """
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    n_ofdm_syms_ref = 50   # reference frame length just for a fair comparison
    for scheme in schemes:
        k = MODULATIONS[scheme]
        payload_bits = k * ofdm.n_data * n_ofdm_syms_ref
        thr, se = compute_throughput_se(scheme, ofdm, n_ofdm_syms_ref, payload_
        axes[0].axhline(thr / 1e6, linestyle="--", alpha=0.5,
                        label=f"{scheme} (fixed)")
        axes[1].axhline(se, linestyle="--", alpha=0.5, label=f"{scheme} (fixed

    adaptive_thr, adaptive_se = [], []
    for snr in snr_range:
        scheme = choose_modulation_adaptive(snr)
        k = MODULATIONS[scheme]
        payload_bits = k * ofdm.n_data * n_ofdm_syms_ref
        thr, se = compute_throughput_se(scheme, ofdm, n_ofdm_syms_ref, payload_
        adaptive_thr.append(thr / 1e6)
        adaptive_se.append(se)

    axes[0].plot(snr_range, adaptive_thr, "k*-", linewidth=2, markersize=8,
                 label="Adaptive")
    axes[0].set_xlabel("SNR (dB)")
    axes[0].set_ylabel("Throughput (Mbps)")
    axes[0].set_title("Throughput vs SNR")
    axes[0].grid(True, alpha=0.3)
    axes[0].legend(fontsize=8)

    axes[1].plot(snr_range, adaptive_se, "k*-", linewidth=2, markersize=8,
                 label="Adaptive")
    axes[1].set_xlabel("SNR (dB)")
    axes[1].set_ylabel("Spectral Efficiency (bits/s/Hz)")
    axes[1].set_title("Spectral Efficiency vs SNR")
    axes[1].grid(True, alpha=0.3)
    axes[1].legend(fontsize=8)

    fig.tight_layout()
    fig.savefig(outpath, dpi=150)
    plt.show()
    plt.close(fig)


def plot_adaptive_scheme_selection(snr_range, outpath: str):
    """Visualize which modulation gets selected across the SNR range."""
    scheme_to_num = {s: i for i, s in enumerate(MOD_ORDER)}
    chosen = [choose_modulation_adaptive(s) for s in snr_range]
    y = [scheme_to_num[s] for s in chosen]

    fig, ax = plt.subplots(figsize=(9, 4))
    ax.step(snr_range, y, where="post", linewidth=2)
    ax.set_yticks(list(scheme_to_num.values()))
    ax.set_yticklabels(list(scheme_to_num.keys()))
    ax.set_xlabel("SNR (dB)")
    ax.set_ylabel("Selected modulation")
    ax.set_title("Adaptive Modulation Selection vs SNR")
    ax.grid(True, alpha=0.3)
```

```
    for th, scheme in ADAPTIVE_THRESHOLDS[:-1]:
        ax.axvline(th, color="red", linestyle=":", alpha=0.6)
    fig.tight_layout()
    fig.savefig(outpath, dpi=150)
    plt.show()
    plt.close(fig)


# ============================================================================
# SECTION 11 : PRETTY PRINTING
# ============================================================================

def print_report(res: OFDMResult):
    print("\n" + "=" * 70)
    print(" TRANSMISSION REPORT")
    print("=" * 70)
    print(f" Modulation scheme      : {res.scheme}  "
          f"({MODULATIONS[res.scheme]} bits/symbol)")
    print(f" Channel SNR (target)   : {res.snr_db:.2f} dB")
    print(f" OFDM symbols sent      : {res.n_ofdm_syms}")
    print(f" FFT size / CP length   : {res.ofdm.n_fft} / {res.ofdm.n_cp}")
    print(f" Data / Pilot subcarriers: {res.ofdm.n_data} / {res.ofdm.n_pilot}"
    print("-" * 70)
    print(f" Original text          : {res.original_text!r}")
    print(f" Recovered text         : {res.recovered_text!r}")
    print(f" Perfect text match     : {res.text_match}")
    print("-" * 70)
    print(f" FEC (repetition code)  : "
          f"{'ON, rate 1/' + str(res.rep) if res.use_fec else 'OFF'}")
    print(f" Raw channel Bit Errors : {res.n_bit_err} / {len(res.tx_bits)} cod
    print(f" Raw channel BER        : {res.ber:.6f}")
    print(f" Symbol Errors          : {res.n_sym_err}")
    print(f" SER                    : {res.ser:.6f}")
    if res.use_fec:
        print(f" Post-FEC Bit Errors    : {res.post_fec_err} message bits")
        print(f" Post-FEC BER           : {res.post_fec_ber:.6f}")
    print(f" EVM                    : {res.evm_percent:.3f} %")
    print(f" Throughput             : {res.throughput_bps/1e6:.4f} Mbps")
    print(f" Spectral Efficiency    : {res.spectral_eff:.4f} bits/s/Hz")
    print("=" * 70 + "\n")


# ============================================================================
# SECTION 12 : MAIN / INTERACTIVE DEMO
# ============================================================================

def safe_input(prompt: str, default: str) -> str:
    """input() that gracefully falls back to a default value when stdin
    is not interactive (e.g. running non-interactively / piped / redirected)."
    try:
        val = input(prompt)
        return val.strip() if val.strip() != "" else default
    except (EOFError, OSError):
        print(f"{prompt}{default}  (no interactive stdin -> using default)")
        return default
```

```
def main():
    parser = argparse.ArgumentParser(description="Adaptive OFDM communication s
    parser.add_argument("--text", type=str, default=None, help="message to sen
    parser.add_argument("--snr", type=float, default=None, help="channel SNR i
    parser.add_argument("--mod", type=lambda s: s.strip().upper(), default=Non
                        choices=list(MODULATIONS.keys()) + ["ADAPTIVE"],
                        help="modulation scheme or ADAPTIVE")
    parser.add_argument("--outdir", type=str, default="ofdm_outputs",
                            help="folder to save plots")
    parser.add_argument("--trials", type=int, default=12,
                            help="Monte-Carlo trials per SNR point in the sweep")
    parser.add_argument("--skip-sweep", action="store_true",
                            help="skip the full BER/SER/EVM vs SNR sweep (faster)
    parser.add_argument("--no-fec", action="store_true",
                            help="disable the repetition-code FEC on the live demo
                                 "(shows raw, uncoded channel performance instead
    parser.add_argument("--rep", type=int, default=5,
                            help="repetition factor for the demo FEC (default 5)"
    args, _unknown = parser.parse_known_args()

    os.makedirs(args.outdir, exist_ok=True)

    print("\n" + "#" * 70)
    print("#  ADAPTIVE OFDM WIRELESS COMMUNICATION SYSTEM - SIMULATION DEMO")
    print("#" * 70)

    # ---------------- user inputs ----------------
    text = args.text if args.text is not None else safe_input(
        "\nEnter the text message to transmit: ", "Hello, Adaptive OFDM World!

    snr_str = str(args.snr) if args.snr is not None else safe_input(
        "Enter channel SNR in dB (e.g. 18): ", "18")
    snr_db = float(snr_str)

    mod_choice = args.mod if args.mod is not None else safe_input(
        "Choose modulation [BPSK/QPSK/16QAM/64QAM/ADAPTIVE]: ", "ADAPTIVE")
    mod_choice = mod_choice.strip().upper()

    ofdm = OFDMSystem(n_fft=64, n_cp=16, pilot_spacing=4, fs=20e6)

    adaptive = (mod_choice == "ADAPTIVE")
    scheme = choose_modulation_adaptive(snr_db) if adaptive else mod_choice
    if scheme not in MODULATIONS:
        print(f"Unknown modulation '{scheme}', defaulting to QPSK.")
        scheme = "QPSK"

    if adaptive:
        print(f"\n[Adaptive Modulation] Operating SNR = {snr_db:.1f} dB "
              f"-> selected scheme = {scheme}")

    # ---------------- single end-to-end run ----------------
    res = run_ofdm_system(text, scheme=scheme, snr_db=snr_db, ofdm=ofdm,
                          num_taps=6, seed=None, use_fec=(not args.no_fec),
                          rep=args.rep)
    print_report(res)
    if not res.text_match:
```

```
            print("NOTE: The current channel realization contains a deep fade on "
                "one or more subcarriers. This is normal behavior in a "
                "frequency-selective Rayleigh fading channel and can increase "
                "the BER for this transmission.\n"
                "For improved recovery, try increasing --snr, selecting a "
                "lower-order modulation, or increasing --rep.\n")

        const_path = os.path.join(args.outdir, "constellation.png")
        chan_path = os.path.join(args.outdir, "channel_response.png")
        plot_constellation(res, const_path)
        plot_channel_response(res, chan_path)
        print(f"Saved: {const_path}")
        print(f"Saved: {chan_path}")

        # ---------------- adaptive-modulation selection curve ----------------
        snr_range_full = np.arange(-5, 31, 1)
        adapt_sel_path = os.path.join(args.outdir, "adaptive_selection.png")
        plot_adaptive_scheme_selection(snr_range_full, adapt_sel_path)
        print(f"Saved: {adapt_sel_path}")

        thr_path = os.path.join(args.outdir, "throughput_se.png")
        plot_throughput_se(snr_range_full, list(MODULATIONS.keys()), ofdm, thr_pat
        print(f"Saved: {thr_path}")

        # ---------------- full Monte-Carlo BER/SER/EVM sweep ----------------
        if not args.skip_sweep:
            print("\nRunning Monte-Carlo BER/SER/EVM sweep across SNR "
                  f"(-5 to 30 dB, {args.trials} trials/point)... this may take a b
            snr_sweep = np.arange(-5, 31, 2.5)
            sweep_results = run_ber_sweep(snr_sweep, list(MODULATIONS.keys()),
                                          n_trials=args.trials, n_bits=4000, ofdm=
            adaptive_curve = build_adaptive_curve(snr_sweep, sweep_results)

            sweep_path = os.path.join(args.outdir, "ber_ser_evm_sweep.png")
            plot_ber_ser_evm_sweep(snr_sweep, sweep_results, adaptive_curve, sweep_
            print(f"Saved: {sweep_path}")
        else:
            print("\n(Skipping BER/SER/EVM sweep as requested.)")

        print("\nAll done. Plots saved in:", os.path.abspath(args.outdir))

        if not IN_NOTEBOOK:
            print(
                "\nNOTE: The simulation completed successfully.\n"
                "Plots have been saved as PNG files in the output folder.\n"
                "If plots are not displayed inline, open the saved PNG files\n"
                "from the output directory to view them.\n\n"
                f"Output folder: {args.outdir}\n"
            )

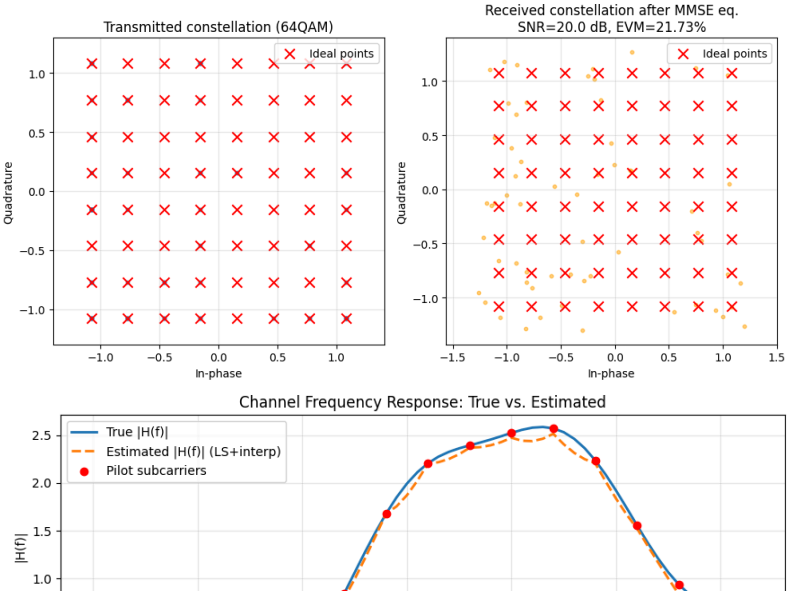

    if __name__ == "__main__":
        main()
```

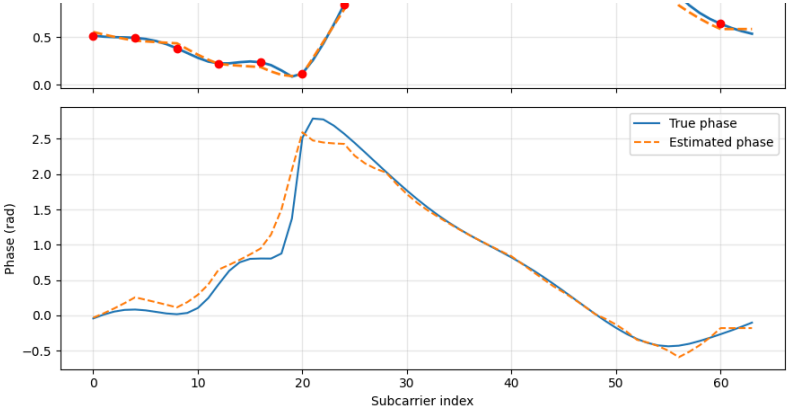
```
##########################################################################
#  ADAPTIVE OFDM WIRELESS COMMUNICATION SYSTEM - SIMULATION DEMO
##########################################################################

Enter the text message to transmit: Hello
Enter channel SNR in dB (e.g. 18): 20
Choose modulation [BPSK/QPSK/16QAM/64QAM/ADAPTIVE]: ADAPTIVE

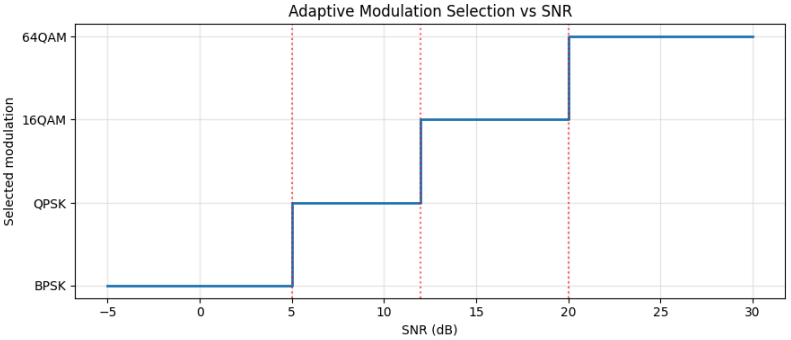
[Adaptive Modulation] Operating SNR = 20.0 dB -> selected scheme = 64QAM

========================================================================
 TRANSMISSION REPORT
========================================================================
 Modulation scheme      : 64QAM  (6 bits/symbol)
 Channel SNR (target)   : 20.00 dB
 OFDM symbols sent      : 2
 FFT size / CP length   : 64 / 16
 Data / Pilot subcarriers: 48 / 16
------------------------------------------------------------------------
 Original text          : 'Hello'
 Recovered text         : 'Hello'
 Perfect text match     : True
------------------------------------------------------------------------
 FEC (repetition code)  : ON, rate 1/5
 Raw channel Bit Errors : 34 / 360 coded bits
 Raw channel BER        : 0.094444
 Symbol Errors          : 21
 SER                    : 0.350000
 Post-FEC Bit Errors    : 2 message bits
 Post-FEC BER           : 0.027778
 EVM                    : 21.728 %
 Throughput             : 5.0000 Mbps
 Spectral Efficiency    : 0.2500 bits/s/Hz
========================================================================
```

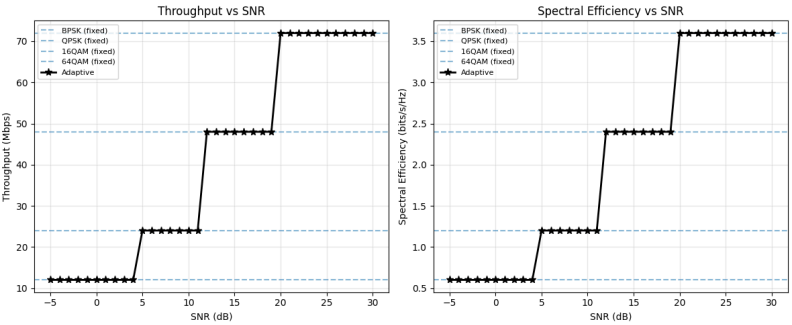





```
Saved: ofdm_outputs/constellation.png
Saved: ofdm_outputs/channel_response.png
```

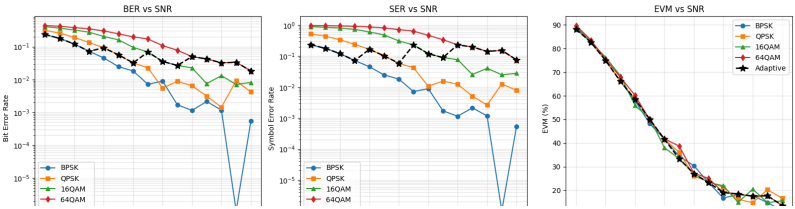


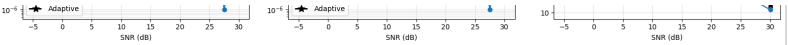

```
Saved: ofdm_outputs/adaptive_selection.png
```




```
Saved: ofdm_outputs/throughput_se.png

Running Monte-Carlo BER/SER/EVM sweep across SNR (-5 to 30 dB, 12 trials/point
```






```
Saved: ofdm_outputs/ber_ser_evm_sweep.png

All done. Plots saved in: /content/ofdm_outputs
```